

What a self-playing population learns, what it forgets, and which of the two you measure

Roland Loechli

Draft, October 3, 2026

Abstract

In a self-play genetic algorithm, the individual exported as “the champion” is close to a median member of its own population, and much of what looks like coevolutionary instability in its learning curve is noise added by the rule that picks it. We show this for David Ha’s tournament-selection algorithm on Slime Volleyball, run as a designed experiment rather than a single run: 59 independent runs of 500,000 self-play games, in a compiled environment that reproduces the reference implementation bit for bit, with every agent scored against a frozen opponent never seen in training. Ha’s export rule promotes the individual with the longest winning lineage; scored against its whole population, that individual ranks on average 60 of 128, its lineage counter is uncorrelated with its skill ($\rho = +0.06$), and the best member of the same pool is 0.90 points per episode better. The population itself is not cycling: skill is almost perfectly transitive across checkpoints. Internal improvement precedes any external transfer by tens of thousands of games, and the timing of the phase change varies 7.5-fold across seeds. An archive of past champions used as a parent abolishes learning; used as a test, it neither harms nor helps detectably, because past champions soon stop being able to win. These claims held in a preregistered replication on 36 fresh runs. Algorithm families differ in reliability, not ceiling. A two-population variant with unequal capacity is reported as exploratory. Any evolutionary loop that promotes one artefact out of a population, including LLM-driven program evolution, has the same exposure; we state this as a hypothesis and give the diagnostic.

1 Introduction

Self-play evolution produces competent agents from an entirely internal signal. In the steady-state genetic algorithm that Ha (2020) uses for Slime Volleyball, two members of a population of 128 play one game, the loser is overwritten by a mutated copy of the winner, and that is all: nothing in the loop refers to an absolute standard of play. Run for 500,000 games, it produces an agent that holds the expert policy shipped with the environment to a draw. The learning curve by which this is usually reported, the score of an exported champion against that expert, is also strikingly unstable: competence is reached, lost and regained between adjacent checkpoints.

The textbook explanation is coevolutionary forgetting. Fitness is relative, so a skill that no current opponent punishes is not selected for and can be lost, and the population cycles (Cliff and Miller, 1995; Popovici et al., 2012); the standard remedy is a hall of fame of past champions to test against (Rosin and Belew, 1997). This paper asks where the instability actually lives, and answers with a design rather than a run: 59 runs across conditions that each change one thing, a pre-registered evaluation protocol, and measurements the external learning curve cannot make.

The main finding is that the explanation is wrong here and the instability is mostly a reporting artefact. Ha’s algorithm exports the individual with the longest winning lineage, by design “without

actually computing who is best to save time” (Ha, 2020). Because the lineage counter is inherited by the loser on every replacement, it measures the age of a lineage, not the merit of an individual. Scored against the whole population, the exported champion is a middling member of it, and the population contains individuals almost a point per episode better. Meanwhile a round robin of every checkpoint against every other shows skill to be almost perfectly transitive: later champions beat earlier ones, and the population is not going in circles. In Miconi’s (2009) terms, historical progress is steady; what fluctuates is our estimate of global progress, and it fluctuates because of which individual we chose to measure.

Contributions.

1. The export rule, not coevolution, is the dominant noise source in the reported curve (§3.4); an internal round robin recovers most of the loss cheaply.
2. Internal improvement precedes external transfer by tens of thousands of games, and the phase change is robust in occurrence but not in timing (§3.1, §3.2).
3. An archive used as a parent abolishes learning; used as a test it is indistinguishable from no archive, and the archive’s own win rate shows why (§3.6).
4. Four algorithm families reach a similar ceiling and differ in how reliably they reach it (§3.7).
5. An exploratory two-population block with unequal capacity (§4).
6. A compiled port of the environment, validated bit for bit, and an analysis pipeline in which every number is generated and checked against the raw files (§7).

Related work. Measuring progress in coevolution is an old problem: Cliff and Miller (1995) introduced tests of current individuals against ancestral opponents, Miconi (2009) separated local, historical and global progress, and recent work compares archive-based methods for global progress (Simione and Nolfi, 2020; Nolfi and Pagliuca, 2025) and measures deceptive progress in algorithms that use a hall of fame (Seals and Tauritz, 2026). Memory mechanisms range from the hall of fame (Rosin and Belew, 1997) to Pareto and Nash-based archives (Ficici and Pollack, 2001, 2003); disengagement between unequal populations is studied by Cartledge and Bullock (2004). In symmetric zero-sum games, any game decomposes into a transitive and a cyclic part (Balduzzi et al., 2019), and real games appear to be transitive at the extremes of skill with cycles concentrated in between (Czarnecki et al., 2020). On the evaluation side, single-seed results in reinforcement learning have repeatedly been shown to mislead (Henderson et al., 2018; Agarwal et al., 2021). What we add is a measurement of the step that all of these take as given: which individual is reported.

2 Setup

Task. Slime Volleyball is a two-player, zero-sum, symmetric game (Ha, 2015), used here through `slimevolleygym` (Ha, 2020). Each agent observes a 12-dimensional state in its own frame and presses 3 binary buttons. An episode ends when a side loses 5 lives or after 3,000 steps; its score is points won minus points lost, in $[-5, 5]$. Every evolved agent is the same fixed 12–10–10–3 tanh network with 273 parameters. Nothing but the weights evolves.

External yardstick. The environment ships a small recurrent policy evolved by Ha (2015): 112 parameters, whose 15 inputs are 8 game observations (its own and the ball’s state, never its opponent’s) and its own previous outputs. It is never used in training. Every checkpoint is scored against it over 200 episodes on one evaluation seed; the final and the best checkpoint of each run are re-scored over 1,000 episodes on a disjoint seed, because the best of 100 checkpoints is a selected maximum. All headline numbers are held out. The protocol was fixed before the runs and is recorded with the data.

The control algorithm. A faithful replication of Ha’s tournament-selection GA: population 128, initial weights $\mathcal{N}(0, 0.5^2)$, mutation $\sigma = 0.1$. Each of 500,000 games draws two members; the loser is replaced by a mutant of the winner and inherits the winner’s streak counter, which then grows by one; a tie mutates one player in place. Every 5,000 games the member with the largest streak is exported, giving 100 checkpoints per run.

Conditions. Each condition changes one thing (Table 1). Seed counts differ by design: 6 for every condition that carries a claim, fewer for side conditions that were cut when the design was revised. Every change after the first run started is recorded, with what was known at the time, in a decision log that ships with the data.

Table 1: Conditions analysed. Two-population runs count once per seed.

condition	what it changes relative to the control	runs
control	Ha’s 2020 GA: population 128, $\sigma = 0.1$, champion = longest winning streak	6
hof-eval-v2	archive as test: with $p = 0.25$ the opponent is an archived champion (capacity 512); genes come only from the living pool	6
hof-0.25	archive as parent: with $p = 0.25$ the opponent is an archived champion (capacity 64) whose mutant replaces a member it beats	6
hof-0.50	archive as parent: with $p = 0.5$ the opponent is an archived champion (capacity 64) whose mutant replaces a member it beats	1
hof-full	archive as parent: with $p = 0.25$ the opponent is an archived champion (capacity 512) whose mutant replaces a member it beats	3
ga2015	generational GA: population 100, 10 games per individual per generation, top 20 kept, uniform crossover, $\sigma = 0.1$	6
es	self-play OpenAI-ES: 50 mirrored perturbations, 4 games each, learning rate 0.03, $\sigma = 0.1$; reports the mean	6
sigma-0.05	mutation scale $\sigma = 0.05$	3
sigma-0.20	mutation scale $\sigma = 0.2$	3
pop-32	population 32	1
asym1x	two populations of 128, 273 v 273 parameters, $\sigma = 0.100$ v 0.100 ; a share 0.25 of games crosses populations	6
asym2x	two populations of 128, 531 v 273 parameters, $\sigma = 0.100$ v 0.100 ; a share 0.25 of games crosses populations	6
asym2x-norm	two populations of 128, 531 v 273 parameters, $\sigma = 0.072$ v 0.100 ; a share 0.25 of games crosses populations	6

Measurements. Beyond the learning curve, three measurements need no external opponent: (i) a round robin of a run’s checkpoints, spaced 50,000 games apart, with Bradley–Terry ratings (Bradley and Terry, 1952) on the Elo scale and a count of cyclic triads; (ii) for the control, which snapshots its

whole population every 50,000 games, every member is scored against the yardstick and compared with the exported one; (iii) the internal transition, the first point at which the population’s own training games last more than 1,500 steps. Comparisons use the exact two-sided Mann–Whitney test with full enumeration (Mann and Whitney, 1947) and Cliff’s δ as effect size; with 6 runs a side the smallest attainable p is 0.002. Secondary comparisons are uncorrected for multiplicity. Because a run that never learns has no volatility, every stability comparison is also reported over the runs that learned.

3 Results

Table 2: Headline results. Points per episode against the 2015 policy, mean $\pm$ s.e.m. across runs, held out. “Learned to rally”: the population ever held 1,500-step rallies against itself. The last two rows are the single run on the unmodified environment and Ha’s published value.

condition	runs	learned to rally	best champion (held out)	end-of-run champion	checkpoints above parity
control (Ha 2020 GA)	6	6/6	$+0.32 \pm 0.06$	-0.15 ± 0.27	26%
archive as test, full span	6	5/6	-0.55 ± 0.86	-1.01 ± 0.83	18%
archive as parent, $p=0.25$	6	1/6	-3.94 ± 0.90	-4.10 ± 0.74	2%
archive as parent, $p=0.50$	1	0/1	$-4.84 \pm \text{—}$	$-4.84 \pm \text{—}$	0%
archive as parent, full span	3	0/3	-4.83 ± 0.01	-4.84 ± 0.00	0%
generational GA (Ha 2015)	6	5/6	-0.68 ± 0.83	-2.00 ± 0.82	3%
self-play ES	6	4/6	-1.56 ± 0.99	-2.08 ± 0.94	7%
sigma = 0.05	3	3/3	$+0.34 \pm 0.07$	-0.17 ± 0.23	26%
sigma = 0.20	3	2/3	-1.44 ± 1.70	-1.66 ± 1.60	11%
population 32	1	0/1	$-4.84 \pm \text{—}$	$-4.86 \pm \text{—}$	0%
<i>reference run, unmodified environment</i>	1	1/1	$+0.50 \pm 0.03$	$+0.04 \pm 0.02$	22%
<i>Ha (2020), same algorithm and budget</i>	1	—	$+0.35 \pm 0.02$	—	—

3.1 Internal improvement precedes external transfer

The reference run, Ha’s algorithm at Ha’s budget on the unmodified environment, has two regimes (Figure 1). At first its champion loses almost every point to the 2015 policy; then it climbs steeply and starts producing champions that beat it. The internal signal moves first: the population’s own rallies pass 1,500 steps at 104,200 games, and the first champion above parity appears at 172,000, a lag of 67,800 games. An evaluation stopped in between reports total failure from a population that is already improving. Competence here means not losing: after the transition the modal outcome is a draw at the step limit, and the final champion wins 19%, draws 64% and loses 17% of held-out episodes (Appendix A.1). Held out, the best checkpoint scores $+0.50$ and the final one $+0.04$, against Ha’s published $+0.35$ for the same algorithm and budget.

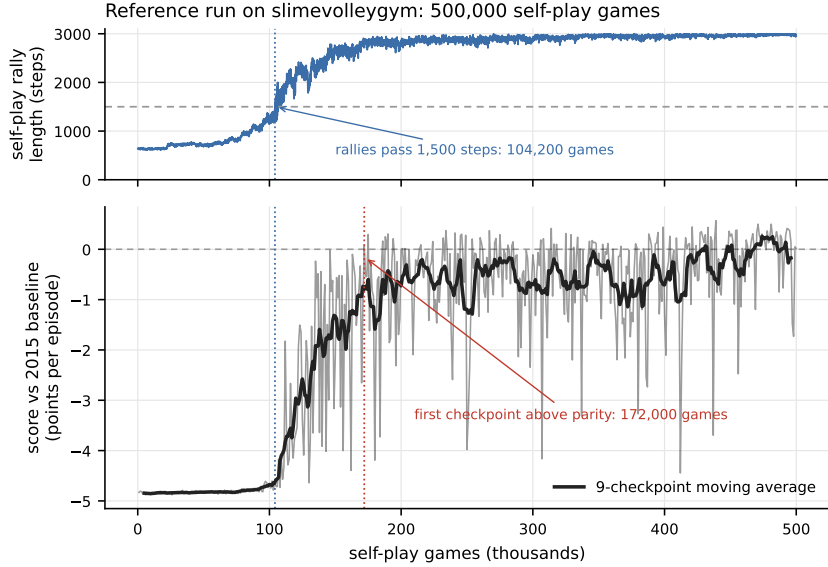


Figure 1: The reference run. Top: rally length of the population’s training games against itself. Bottom: score of the exported champion against the 2015 policy, every checkpoint (thin) and a moving average (thick). Dotted lines mark the internal transition and the first checkpoint above parity.

3.2 The phase change is robust; its timing is not

Across 6 control seeds the internal transition happens anywhere from 55,000 to 415,000 games, a 7.5-fold spread, and the first above-parity checkpoint from 85,000 to 440,000 (Figure 2). The lag between them is always present, 25,000 to 100,000 games. Every seed reaches the same band (6/6 learned to rally, 6/6 produced an above-parity champion), so the occurrence of the transition replicates and its timing does not. A single run supports no claim about when self-play starts working, although the first version of this study, with one seed, reported a single transition time. Nor does the swing settle once the level rises: in 100,000-game windows the control’s mean climbs from -4.32 to -0.32 , while the spread of checkpoint scores within a run stays between 0.51 and 0.83 (Appendix A.1).

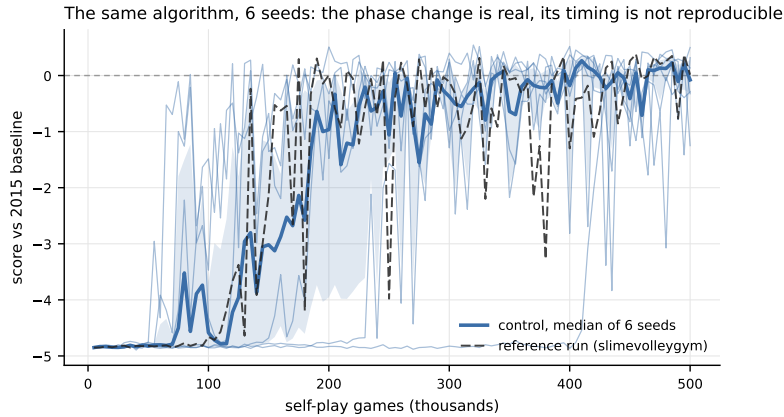


Figure 2: Every control seed (thin), median and inter-quartile band (thick), and the reference run on the same checkpoint grid (dashed).

3.3 The population is not cycling

If the swings were forgetting, later champions should not reliably beat earlier ones, and the checkpoint tournament should contain cycles. Neither is the case (Figure 3). Within control runs the rank correlation between Elo and training time averages $+0.74$, and 0.2% of decided checkpoint triads are cyclic; no condition whose runs all learned exceeds 0.2% . Only the archive-as-parent runs, which never learned, show substantial intransitivity: their champions are equally bad policies whose pairwise results are noise. The game, at the scale of skill explored here, is dominated by its transitive component (Balduzzi et al., 2019; Czarnecki et al., 2020).

The checkpoints are 50,000 games apart, so a faster cycle would be invisible. Played at 5,000-game spacing (all 100 champions of each control run, 20 games per pair; Appendix A.7), the same deadband rule finds 1.0% of decided triads cyclic, but at that sample size the deadband lets noise decide pairs between near-equal policies. When a pair counts as decided only if an exact sign test on its wins and losses says so, 35 of 273,376 decided triads are cyclic (0.01%): not cycling at the finer resolution either.

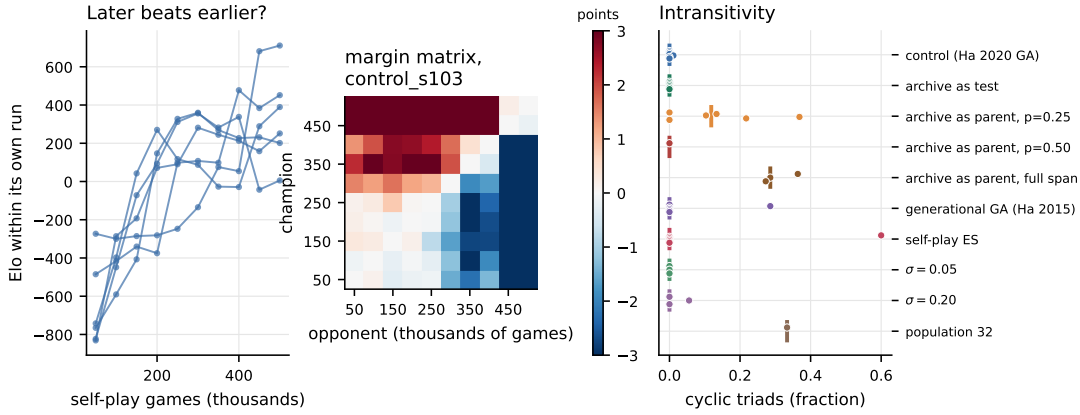


Figure 3: Left: Elo of each checkpoint within its own run, control seeds. Middle: pairwise margin matrix of one control run, later checkpoints against earlier. Right: share of decided checkpoint triads that are cyclic, per condition.

3.4 The export rule is the noise source

Because the control snapshots its whole population, the exported champion can be compared with every individual it was chosen from (Figure 4). Over 6 seeds and all snapshots, the exported individual ranks on average 60 of 128 in its own pool. The rank correlation between the streak counter and actual skill is $+0.06$, and the best member of the same pool is 0.90 points per episode better. At the last snapshot the exported champion averages -0.08 while 67 of its peers are above parity. The reported curve also loses competence that the population does not: summed over all control runs, the exported champion’s score fell by 8.6 points between consecutive snapshots, the best member of the same pools by 1.1 and the median member by 0.9. The population almost never gets worse; the individual we report does. The counter cannot do better: it is inherited on every replacement, so it records how long a lineage has survived, not how good its current member is. In a separate re-scoring of the same snapshots, the streak rule (-1.95) does no better than picking the population’s median member (-1.83); the best member averages -1.14 .

The rule can be replaced without external information. Ranking the pool by an internal round robin of 4,096 games per snapshot, 0.8% of the training budget, closes 65% of the gap between the

exported and the best member (Appendix, Table 6). Its correlation with true skill saturates near +0.57: internal fitness measures skill against the current population, which is not the same as skill against an unseen opponent, so held-out evaluation remains necessary for selection as well as for reporting.

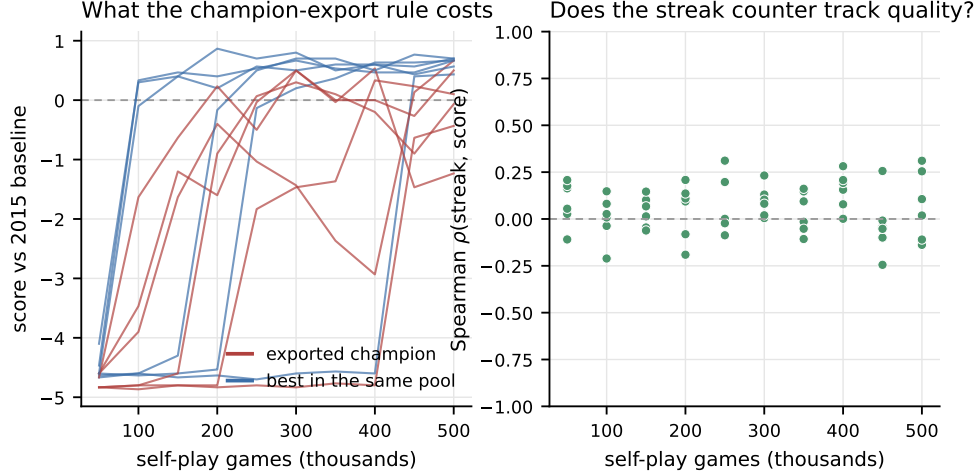


Figure 4: Left: the exported champion (red) and the best member of the same population (blue), every control seed. Right: Spearman correlation between the streak counter and score against the 2015 policy, every seed and snapshot.

3.5 The mutation step is not the cause

A third explanation for the swings is a mutation step too large for the landscape (Appendix A.3). Halving it changes nothing that matters: 3/3 runs learned, 26% of checkpoints are above parity (control 26%), and the volatility of the runs that learned matches the control’s ($\delta = +0.00$, $p = 1.000$). Doubling it costs runs (2/3 learned) but not stability ($\delta = -0.17$, $p = 0.857$ among the runs that learned). Over all runs the larger step even looks calmest (volatility 0.38 against 0.67), because a run that never learns contributes a flat line, which is why stability is only compared among runs that learned. A population of 32, run once, never learned (0/1).

3.6 An archive of past champions: as a parent, and as a test

The standard remedy for forgetting is a hall of fame (Rosin and Belew, 1997). We tested two readings of it. Our first implementation applied Ha’s replacement rule to archive games, so a winning archived genome became the parent of the member it beat: an archive used as a *parent*. With $p = 0.25$ that copies an old genotype back into the pool in 12.0% of all games, and it abolishes learning: 1/6 runs learned to rally, against 6/6 for the control, and 0/4 at a higher dose or with a full-run archive (Table 2). These runs are also the most “stable” in the matrix, with near-zero volatility and drawdown, which is why stability is only compared among runs that learned.

The literature’s reading uses the archive as a *test*: archived champions are opponents, never parents. In *hof-eval-v2* an archive game that the population member loses costs it its slot, but the replacement comes from the living pool; the archive holds a champion from every 1,000 games of the run (capacity 512). Done this way the archive is neither destructive nor helpful (Figure 5). 5/6 runs learned to rally and 4/6 produced an above-parity champion, against 6/6 and 6/6 for the

control, and at 6 runs a side no outcome is distinguishable from the control: checkpoints above parity 18% against 26% ($\delta = -0.33$, $p = 0.370$), mean level over the last 100,000 games $\delta = -0.22$ ($p = 0.589$), best held-out champion $\delta = +0.00$ ($p = 1.000$). The archive’s own win rate shows why it has little to do. Early in a run, archived champions win 0.46–0.50 of their games; in every run whose population learned they end at 0.11–0.16, so 21–22% of late games produce no selection event (an archive game the member wins overwrites nothing). Skill is transitive (§3.3), so a past champion is simply a weaker player. In the run that never learned, the archive’s win rate stayed at 0.50. Logging that win rate is a cheap check on whether an archive still has anything to teach; here it tracked progress, but it is a description of 6 runs, not a validated predictor. An earlier version of this condition credited a win to the living peer that supplied the replacement genes, although that peer had not played; since the longest streak decides what enters the archive, the bug changed the whole trajectory. Those runs are superseded and excluded. With the bug the archive looked measurably worse than the control and learned later; the corrected runs do not, and the decision log shipped with the data gives both side by side.

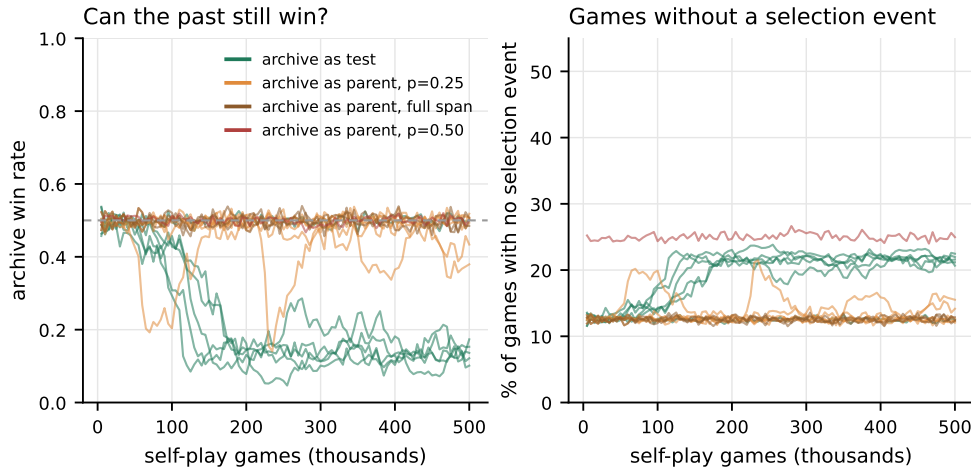


Figure 5: Left: the archive’s win rate against the current population, every archive run. Right: the share of games that produce no selection event, because an archive game the population member wins overwrites nothing.

3.7 Algorithm families differ in reliability, not ceiling

Two further families change the machinery with policy and environment fixed: a generational GA in the style of Ha’s 2015 experiment, which ranks by an explicitly computed fitness and uses crossover, and a self-play evolution strategy (Salimans et al., 2017; Ha, 2017), which reports its distribution mean and so has no export rule at all (Figure 6). With the archive-as-test condition as a fourth family, all four reach a similar ceiling: the single highest end-of-run champion of the study, +0.50, is a self-play ES seed, above every control seed (best +0.41). They differ in their floor. Only the control learned to rally and produced an above-parity champion in every seed (6/6, 6/6); the generational GA managed 5/6 and 5/6, the ES 4/6 and 2/6, the archive as test 5/6 and 4/6. Across seeds, the spread of end-of-run champions is 3.0–3.5 times the control’s for the other three families. A study with one seed per family could have crowned the ES on the strength of a seed that happened to work. The cross-run tournament, in which the final champions of all 41 single-population runs play each other with no external opponent, shows the same floor (Appendix A.5): every control

champion rates above zero (worst +225), while each of the other three families has a champion at or below -576 ; across runs, too, skill is transitive (0.1% of decided triads cyclic). The generational GA ranks by a computed fitness, which §3.4 suggests should help, and still does not come out ahead; crossover between weight vectors and its coarser, generational update are plausible reasons we cannot separate here. The self-play ES was tuned only at pilot scale, so its result is a statement about the configuration we could afford, not about the method.

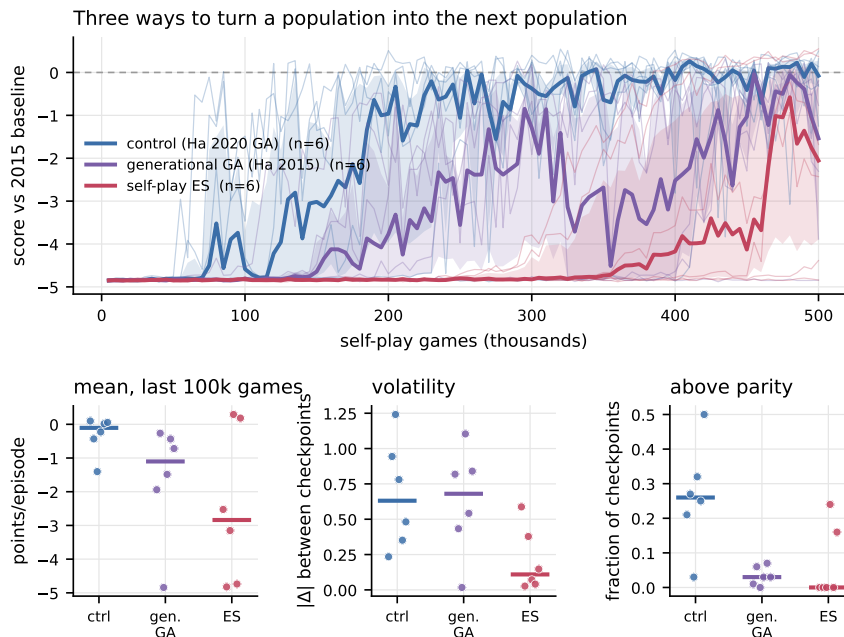


Figure 6: Three families at an identical budget. Top: median and inter-quartile band. Bottom: per-seed level and volatility of the last 100,000 games and the share of checkpoints above parity (ctrl: control; gen. GA: generational GA).

3.8 A preregistered replication

The results above rest on six seeds per main condition, and the design was revised while it ran (Section 6). The claims about Ha’s GA and the archive (Sections 3.1–3.4 and 3.6) were therefore tested again on fresh seeds, with every test, threshold and decision rule committed before the first run: 12 new runs each of the control, the archive as parent and the archive as test, 36 in all, measured with the code that produced the results above and analysed once. 6 of 6 claims replicated (Table 8 in Appendix A.6). The internal transition preceded parity in 10/10 eligible runs; checkpoint skill was transitive ($\rho = +0.79$, 0.5% cyclic triads); the exported individual ranked on average 56 of 128 in its pool, with $\rho(\text{streak}, \text{skill}) = +0.04$, and lost 21.2 points between snapshots where the pool’s best member lost 7.6; the archive as parent learned in 1/12 runs; the archive as test was again indistinguishable from the control (every $p \geq 0.44$).

Two statements come out weaker than six seeds suggested. Not every control run learns: 10/12 did, which meets the preregistered threshold but not “every seed”. And a falling archive win rate does not by itself indicate learning: it fell to 0.06–0.16 in the archive-as-test runs that learned, but also to 0.21 in one that never did. C6 and C7 were not part of the replication.

4 Unequal power (exploratory)

This block was designed after the main matrix had run and re-implemented twice before the runs reported here; it is exploratory and we draw no confirmatory conclusion from it. Two populations of 128 play only each other, a share 0.25 of each side’s games crossing over; in two conditions one side has a 12–16–16–3 network with 531 parameters against 273, in one with a common per-parameter σ and in one with the larger side’s σ scaled to equalise the mutation step norm. A symmetric two-population control has 273 a side (Appendix, Figure 11 and Table 11).

Of 18 runs, 1 ended with both pools holding a member above parity; 10 ended with exactly one, 7 with neither. 13 ended in runaway dominance (one side winning more than a share 0.9 of the cross-population games at the end), 4 of them in the symmetric control, where the sides differ only in their seed. Neither capacity condition is distinguishable from that control (common σ : $\delta = -0.33$, $p = 0.394$; matched step: $\delta = +0.11$, $p = 0.818$). The larger side led in 2/6 runs at common σ and in 5/6 with the step norm matched ($\delta = +0.67$, $p = 0.065$), which suggests, but at 6 seeds a side does not establish, that extra capacity converts into advantage only when the mutation scale is adjusted to it. The pattern of one side collapsing is consistent with disengagement (Cartlidge and Bullock, 2004).

5 Discussion

Three sources of variance, not two. Coevolutionary methodology separates the *ecology*, which selects, from the *yardstick*, which measures. This study found a third component with its own error term: the *promotion rule* that names one artefact out of the population. In the control, the reported curve lost about 8 times as much ground between population snapshots as the population’s best member did, and it does so invisibly, because a champion curve looks the same whether its swings come from the population or from the choice of individual.

A hypothesis about other evolutionary loops. The failure does not depend on skill being transitive. In a synthetic game whose mix of transitive and cyclic skill is set by one parameter, run with the same GA and preregistered (repository, docs/lab.md), the streak rule exported an individual ranked on average 52–63 of 128 in its own pool at every mixture, its streak barely related to its true strength ($\rho = +0.07$ to $+0.13$). Any loop that promotes one artefact out of a population has a promotion rule, and that rule is usually cheaper than a measurement of quality. LLM-driven program evolution (Novikov et al., 2025; Lange et al., 2025) promotes one program per generation or per run on the strength of an evaluator that may be noisy, small or proxy-based. Gideoni et al. (2026) report the closely related failure in that setting: agentic scaffolds selected on small validation sets lose much of their apparent advantage on held-out data, because the selection was partly by chance. We do not test program evolution here. We state as a hypothesis that its reported progress curves contain a promotion-rule component of the kind measured above, and propose the same diagnostic: before attributing a noisy progress curve to the search, check whether the statistic used to promote correlates with held-out quality across the candidates it chose between, and compare the promoted candidate with the population’s median.

Diagnose before applying a remedy. A cheap within-run round robin found skill transitive and so predicted that an archive, the remedy for intransitivity, would have little to do; used as a test it changed nothing measurable, used as a parent it pulled the loop backwards.

6 Limitations

- **One environment**, symmetric, zero-sum and fully observed: the most favourable setting for purely relative selection, and one whose skill structure is largely transitive. In a synthetic game with tunable cyclic skill (repository, preregistered, 96 runs), cycling appeared as designed (13.5% cyclic champion triads at the most cyclic setting), the export rule failed in the same way, and an archive used as a test did not help more as skill became cyclic (trend $p = 0.116$). Neither did an archive organised by behaviour, one champion per cell of a MAP-Elites-style grid (repository, preregistered, 60 runs: trend $p = 0.072$; at the most cyclic setting it did not beat the time-ordered archive, $p = 0.34$). Neither archive design helped. A physical game with cyclic skill remains untested.
- **Few seeds**. 6 runs per condition separate large effects from seed noise, not small ones; several conditions have fewer. Secondary comparisons are uncorrected for multiplicity. C1–C5b were confirmed on 12 fresh runs per condition (§3.8); C6 and C7 were not.
- **Post hoc design changes**. The design was revised while it ran (seed counts, the archive conditions, the replacement of the population sweep by the unequal-power block, the rerun of the archive-as-test condition after a bug fix). Each change is logged with what was known when it was made, but the final design is not the one pre-registered; the replication of §3.8 was.
- **Scope of the measurements**. The export analysis is control-only, because only the control stores population snapshots, and the ES was tuned only at pilot scale. The archive as a parent has one design; as a test it has two: the time-ordered archive of C5b and, in the repository, one organised by behaviour, which in Slime Volleyball also had no detectable effect (all $p \geq 0.29$ against the control on 12 runs per arm, with 37–53 of 64 cells occupied).
- **Fixed topology**. C1–C6 come from families that evolve one fixed network. NEAT (Stanley and Miikkulainen, 2002), which evolves structure, was run in the repository as a preregistered fifth family in the same game, under the generational GA’s evaluation and the same budget. It never learned to rally (0 of 12 runs; final champions -4.84 against the 2015 baseline), detectably worse than the generational GA (Cliff’s $\delta = -0.92$, $p = 0.0008$), while growing 23–39 hidden nodes. This is one configuration with mostly published settings (Stanley and Miikkulainen, 2004), untuned for this game.
- **The unequal-power block is exploratory** and underpowered for its most interesting comparison.

7 Reproducibility

The code, every run file, the protocol, the decision log and this paper’s sources are in the repository. A 500,000-game run would take about 13 core-hours on the reference implementation; the compiled port runs 26 times faster on one core. It is validated by driving both implementations from one stream of serve velocities: 200 paired games agree on every ball and agent position, every rally end, every score and every episode length, over 304,812 environment steps (Appendix, Table 16). Every table and every number in this paper is generated from the analysis files, and every analysis file records the hash of each raw file it was computed from; a consistency check, run on every push, fails if any number does not follow from the files on disk. Rerunning the analysis pipeline reproduces the committed analysis files.

Acknowledgements

The environment, the 2015 baseline policy and the genetic algorithm studied here are David Ha’s (Ha, 2015, 2020), released under the Apache-2.0 licence; the compiled port in this study transcribes them. For background on neuroevolution and coevolution see Risi et al. (2026).

References

- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021.
- David Balduzzi, Marta Garnelo, Yoram Bachrach, Wojciech M. Czarnecki, Julien Pérolat, Max Jaderberg, and Thore Graepel. Open-ended learning in symmetric zero-sum games. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 434–443, 2019.
- Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3–4):324–345, 1952. URL <https://academic.oup.com/biomet/article-abstract/39/3-4/324/326091>.
- John Cartlidge and Seth Bullock. Combating coevolutionary disengagement by reducing parasite virulence. *Evolutionary Computation*, 12(2):193–222, 2004. URL <https://ieeexplore.ieee.org/document/6790490>.
- Dave Cliff and Geoffrey F. Miller. Tracking the red queen: Measurements of adaptive progress in co-evolutionary simulations. In *Advances in Artificial Life (ECAL 1995)*, volume 929 of *Lecture Notes in Computer Science*, pages 200–218. Springer, 1995. doi: 10.1007/3-540-59496-5_300.
- Wojciech M. Czarnecki, Gauthier Gidel, Brendan Tracey, Karl Tuyls, Shayegan Omidshafiei, David Balduzzi, and Max Jaderberg. Real world games look like spinning tops. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- Sevan G. Ficici and Jordan B. Pollack. Pareto optimality in coevolutionary learning. In *Advances in Artificial Life (ECAL 2001)*, volume 2159 of *Lecture Notes in Computer Science*, pages 316–325. Springer, 2001. doi: 10.1007/3-540-44811-X_34.
- Sevan G. Ficici and Jordan B. Pollack. A game-theoretic memory mechanism for coevolution. In *Genetic and Evolutionary Computation (GECCO 2003)*, volume 2723 of *Lecture Notes in Computer Science*, pages 286–297. Springer, 2003. doi: 10.1007/3-540-45105-6_35.
- Yonatan Gideoni, Sebastian Risi, and Yarin Gal. Simple baselines are competitive with code evolution, 2026.
- David Ha. Neural slime volleyball. Blog post, <https://blog.otoro.net/2015/03/28/neural-slime-volleyball/>, 2015.
- David Ha. ESTool: Evolution strategies tool. <https://github.com/hardmaru/estool>, 2017.
- David Ha. Slime volleyball gym environment. <https://github.com/hardmaru/slimevolleygym>, 2020.

- Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. doi: 10.1609/aaai.v32i1.11694.
- Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. ShinkaEvolve: Towards open-ended and sample-efficient program evolution, 2025.
- Henry B. Mann and Donald R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1):50–60, 1947. doi: 10.1214/aoms/1177730491.
- Thomas Miconi. Why coevolution doesn’t “work”: Superiority and progress in coevolution. In *Genetic Programming (EuroGP 2009)*, volume 5481 of *Lecture Notes in Computer Science*, pages 49–60. Springer, 2009. doi: 10.1007/978-3-642-01181-8_5.
- Stefano Nolfi and Paolo Pagliuca. Global progress in competitive co-evolution: A systematic comparison of alternative methods. *Frontiers in Robotics and AI*, 2025. doi: 10.3389/frobt.2024.1470886.
- Alexander Novikov et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025.
- Elena Popovici, Anthony Bucci, R. Paul Wiegand, and Edwin D. de Jong. Coevolutionary principles. In *Handbook of Natural Computing*, pages 987–1033. Springer, 2012. doi: 10.1007/978-3-540-92910-9_31.
- Sebastian Risi, Yujin Tang, David Ha, and Risto Miikkulainen. *Neuroevolution: Harnessing Creativity in AI Agent Design*. MIT Press, 2026. ISBN 9780262054768. Open-access online edition at <https://neuroevolutionbook.com>.
- Christopher D. Rosin and Richard K. Belew. New methods for competitive coevolution. *Evolutionary Computation*, 5(1):1–29, 1997. URL <https://direct.mit.edu/evco/article-abstract/5/1/1/790/New-Methods-for-Competitive-Coevolution>.
- Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning, 2017.
- Deacon Seals and Daniel R. Tauritz. Tripping over the past: Measuring deceptive progress in competitive coevolutionary algorithms employing hall of fame. In *Applications of Evolutionary Computation (EvoApplications 2026)*, Lecture Notes in Computer Science. Springer, 2026. doi: 10.1007/978-3-032-23607-4_32.
- Luca Simione and Stefano Nolfi. Long-term progress and behavior complexification in competitive coevolution. *Artificial Life*, 26(4):409–430, 2020. URL <https://direct.mit.edu/artl/article/26/4/409/97302>.
- Kenneth O. Stanley and Risto Miikkulainen. Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127, 2002. URL <https://direct.mit.edu/evco/article/10/2/99/1123/Evolving-Neural-Networks-through-Augmenting>.
- Kenneth O. Stanley and Risto Miikkulainen. Competitive coevolution through evolutionary complexification. *Journal of Artificial Intelligence Research*, 21:63–100, 2004.

A Supplementary results

A.1 The reference run in detail

The best checkpoint of the reference run, at 439,000 games, wins 42%, draws 52% and loses 6% of held-out episodes; the final one wins 19%, draws 64% and loses 17% (Table 3). On the sweep, 112/500 of its checkpoints score above parity. What transfers to the 2015 policy is a general ability not to lose, not a counter-strategy: the population never sees that opponent.

The single-run version of this study reported that the swings shrink as the level rises. With 6 seeds that is testable, and it does not hold (Table 4): from the first to the last 100,000-game window the control’s mean rises from -4.32 to -0.32 and the checkpoints above parity from 1/120 to 64/120, but the within-run spread of checkpoint scores is 0.74 in the first window and 0.72 in the last.

Table 3: The reference run: final and best checkpoint, held out.

checkpoint	games	score vs 2015 baseline	won / drawn / lost	mean rally
final	500,000	$+0.036 \pm 0.734$ (s.e.m. 0.023)	19% / 64% / 17%	3000 steps
peak	439,000	$+0.496 \pm 0.856$ (s.e.m. 0.027)	42% / 52% / 6%	3000 steps
Ha (2020), same algorithm and budget	500,000	$+0.353 \pm 0.728$	—	—

Table 4: The control in 100,000-game windows: mean score across seeds, within-run spread of checkpoint scores, spread of window means across seeds, and checkpoints above parity.

games	mean across seeds	within-run s.d.	spread across seeds	checkpoints above parity
0–100,000	-4.32	0.74	0.57	1/120
100,000–200,000	-2.76	0.83	1.78	14/120
200,000–300,000	-1.46	0.69	1.75	34/120
300,000–400,000	-1.01	0.51	1.71	45/120
400,000–500,000	-0.32	0.72	0.52	64/120

A.2 The export rule over training, and promotion rules

Table 5 follows the comparison of §3.4 through training: at every snapshot the exported individual sits in the middle of its population, the streak counter’s correlation with skill stays near zero, and the gap to the best member of the pool persists. Table 6 and Figure 7 replace the streak rule with an internal round robin of increasing size; the rule uses no information the algorithm does not already have, so it is deployable, unlike the best-in-pool oracle.

Table 5: The exported champion against its own population, control runs, averaged across seeds at each snapshot.

games	exported champion	best in the same pool	gap	exported rank	$\rho(\text{streak, score})$	above parity in pool	mean pairwise genotype distance
50,000	-4.73	-4.51	0.22	50 / 128	+0.09	0	12.66
100,000	-3.91	-2.22	1.69	71 / 128	+0.00	1	10.79
150,000	-2.94	-2.05	0.89	58 / 128	+0.04	5	10.43
200,000	-2.05	-1.31	0.74	54 / 128	+0.05	13	10.13
250,000	-1.36	-0.42	0.93	50 / 128	+0.07	23	9.07
300,000	-1.07	-0.29	0.78	44 / 128	+0.10	32	10.50
350,000	-1.41	-0.31	1.09	73 / 128	+0.04	34	10.53
400,000	-1.18	-0.30	0.88	56 / 128	+0.15	47	12.61
450,000	-0.48	+0.54	1.03	76 / 128	-0.03	49	12.23
500,000	-0.08	+0.62	0.70	61 / 128	+0.07	67	11.38

Table 6: Promotion rules on the control’s population snapshots. “Gap closed”: share of the distance between the streak-exported and the best member recovered by the rule.

promotion rule	games spent ranking	mean score of the exported individual	volatility of the reported series	gap to best closed	$\rho(\text{rule statistic, true skill})$
winning streak (Ha’s rule)	0	-1.95	0.84	—	+0.04
<i>pick the population median</i>	0	-1.83	0.57	—	—
internal round robin, 4 peers each	256	-1.77	0.72	22%	+0.32
internal round robin, 8 peers each	512	-1.55	0.77	50%	+0.41
internal round robin, 16 peers each	1,024	-1.49	0.70	57%	+0.49
internal round robin, 32 peers each	2,048	-1.47	0.73	60%	+0.53
internal round robin, 64 peers each	4,096	-1.43	0.68	65%	+0.57
<i>best in pool (oracle, not deployable)</i>	—	-1.14	0.61	100%	+1.00

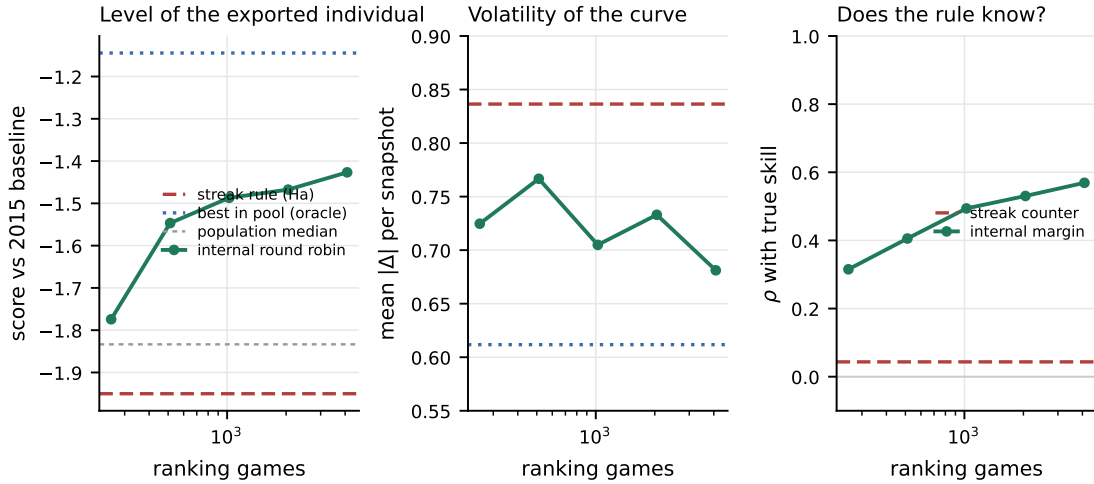


Figure 7: Promotion rules as a function of the games spent ranking: level of the exported individual, volatility of the reported curve, and rank correlation between the rule’s statistic and true skill.

A.3 Mutation scale and population size

Figure 8 shows the sweeps discussed in §3.5. The population sweep was cut to a single run of the smaller population when the unequal-power block replaced it; the larger population was never run.

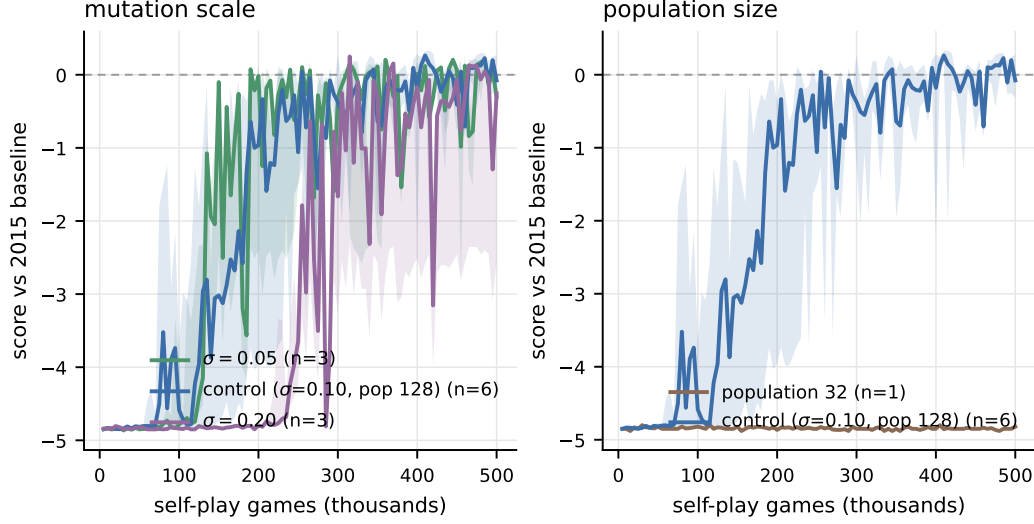


Figure 8: Mutation scale (left) and population size (right) against the control, median and inter-quartile band across seeds.

A.4 The archive conditions

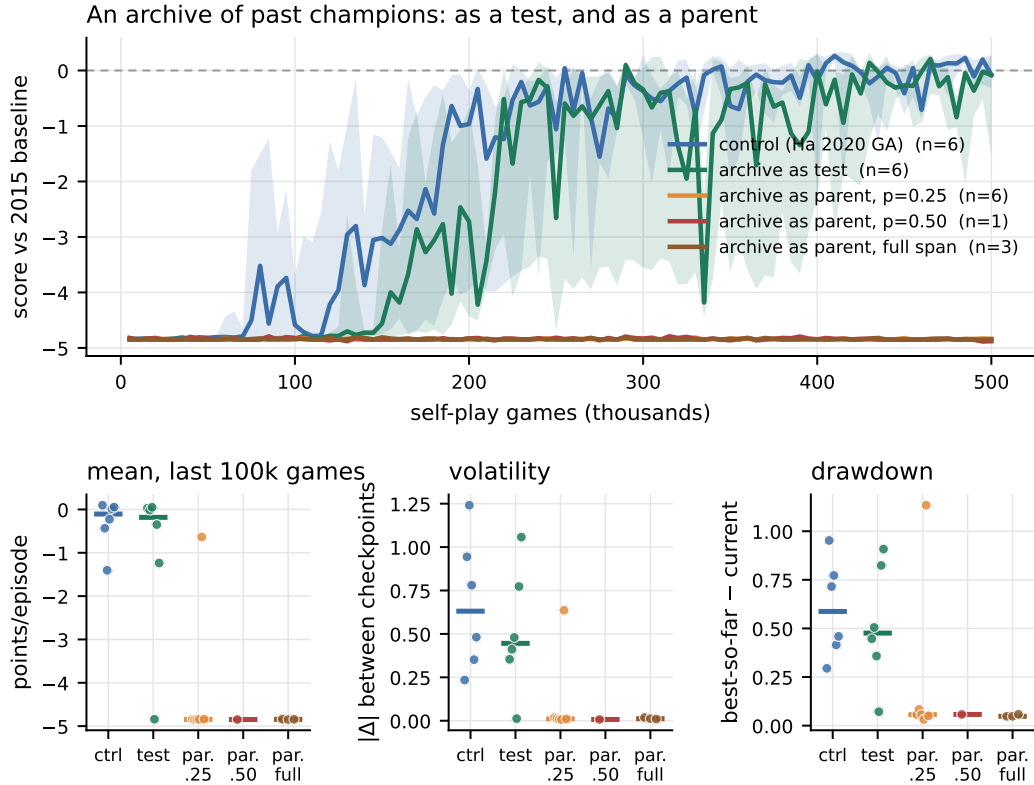


Figure 9: The archive conditions against the control. Top: median and inter-quartile band. Bottom: per-seed level, volatility and drawdown of the last 100,000 games (ctrl, control; test, archive as test; par., archive as parent at the dose or span given in the legend).

A.5 The cross-run tournament

Scores against a single frozen opponent can reward specialising against it. The final champion of every single-population run therefore played the final champion of every other (Figure 10, Table 7). The control’s median rating is +295 and its worst champion +225; the archive as test has a higher median (+381) but also a champion at -596 ; the generational GA and the ES have medians of +56 and +99. The highest median belongs to the smaller mutation step (+472), a side condition with fewer runs.

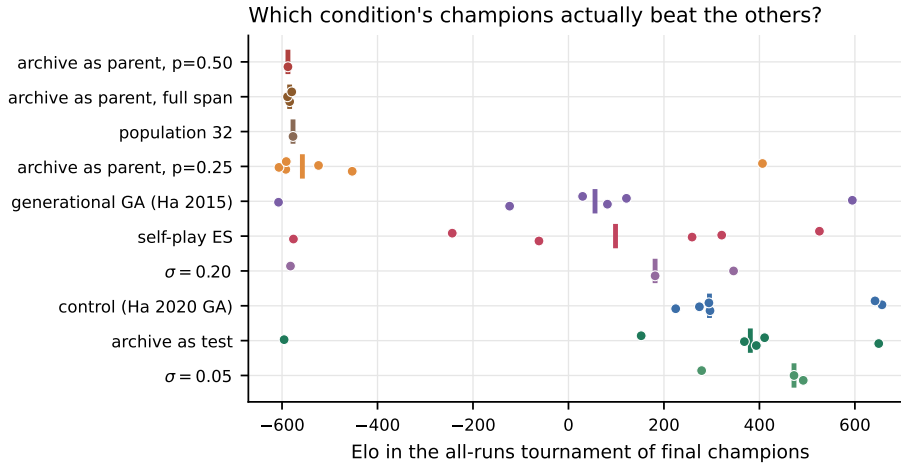


Figure 10: Bradley–Terry ratings on the Elo scale of every run’s final champion in one all-play-all tournament; bars mark condition medians.

Table 7: Cross-run tournament of final champions, by condition.

condition	runs	median Elo	best run	worst run
control (Ha 2020 GA)	6	+295	+657	+225
archive as test, full span	6	+381	+650	-596
archive as parent, $p=0.25$	6	-557	+406	-606
archive as parent, $p=0.50$	1	-588	-588	-588
archive as parent, full span	3	-584	-580	-589
generational GA (Ha 2015)	6	+56	+595	-607
self-play ES	6	+99	+526	-576
$\sigma = 0.05$	3	+472	+492	+279
$\sigma = 0.20$	3	+181	+346	-582
population 32	1	-577	-577	-577

A.6 The preregistered replication

The design, hypotheses, tests and decision rules are in `results/replication/PREREGISTRATION.md`, committed before the first replication run; `replication.py` is the analysis, run once. Its measurements are the main text’s own and, applied to the original seeds, reproduce the committed analysis files bit for bit. One run was trained in the session’s container and the rest on GitHub’s runners, after the container was reclaimed twice mid-run; a run repeated on both machines gave identical files (`results/matrix/decisions.md`).

Table 8: The preregistered replication: 12 fresh runs per condition, every criterion fixed before the first run. Tests in the Holm family hold when rejected at family-wise $\alpha = 0.05$.

claim	criterion	result	holds
C1	internal transition before parity (lag > 0)	10/10 runs, $p = 9.8\text{e-}04$	yes
C2	control runs that learn to rally	10/12	yes
C2	latest / earliest internal transition ≥ 3	5.8×	yes
C3	$\rho(\text{Elo, time}) > 0$ within runs	12/12 runs (mean +0.79), $p = 2.4\text{e-}04$	yes
C3	cyclic share of decided triads < 1%	4/849 (0.5%)	yes
C4	exported individual outside its pool's top quarter	12/12 runs (mean rank 56), $p = 2.4\text{e-}04$	yes
C4	$\rho(\text{streak, skill})$ inside ± 0.2 (90% CI)	+0.04 [+0.02, +0.06]	yes
C4	exported declines more than the best member	10/12 runs (21.2 vs 7.6), $p = 0.019$	yes
C5a	archive as parent learns less often than control	1/12 vs 10/12, $p = 3.2\text{e-}04$	yes
C5b	archive as test vs control: all four $p \geq 0.05$	final $\delta +0.01$ p 0.98, peak $\delta -0.19$ p 0.44, above $\delta -0.15$ p 0.56, late $\delta -0.08$ p 0.76	yes

A.7 Transitivity at 5,000-game spacing

Table 9 repeats the within-run tournament of Section 3.3 on every stored champion of the control runs, one per 5,000 games, with two rules for when a pair counts as decided. Under the main text’s deadband rule the pooled cyclic share is 1.0%, and 4.5% among triads spanning at most 50,000 games; under an exact sign test it is 0.01%. The difference is sampling noise: with 20 games per pair, the margin between two near-equal policies has a standard error of the order of the deadband. The same tournament shows the exported champion moving at this resolution while the population does not: among adjacent champions whose difference the sign test decides, the later one wins 88/144 times (61%).

Table 9: Within-run tournament at 5,000-game spacing, control runs.

run	$\rho(\text{Elo, time})$	cyclic, ± 0.25 rule	cyclic, sign test	within 50k games, sign test	next beats previous
control_s101	+0.95	0.69% (774/112,190)	0.00% (0/51,268)	0.00% (0/272)	10/14
control_s102	+0.76	0.55% (541/98,864)	0.02% (11/51,840)	0.23% (1/430)	18/35
control_s103	+0.81	3.82% (3,675/96,104)	0.00% (1/21,569)	0.00% (0/198)	7/13
control_s104	+0.68	0.50% (431/86,322)	0.04% (16/43,215)	0.23% (1/440)	21/34
control_s105	+0.91	0.22% (241/108,654)	0.00% (1/58,693)	0.00% (0/454)	16/24
control_s106	+0.85	0.30% (253/85,284)	0.01% (6/46,791)	0.86% (3/349)	16/24
<i>all control runs</i>	—	1.01% (5,915/587,418)	0.01% (35/273,376)	0.23% (5/2,143)	88/144

A.8 Stronger external opponents

The 2015 baseline never sees its opponent, so “above parity” against it is a low bar. Ha (2020) also publishes two trained policies: a self-play GA champion of the same 273-parameter class, and a 743-parameter CMA-ES policy trained against the baseline, which score +0.35 and +1.15 against it. Every final champion played 200 games against each, half on each side, on the compiled environment; for networks of both sizes it reproduces the reference environment bit for bit (40 games, 120,000 steps compared).

The baseline ranks the final champions much as the stronger opponents do (Spearman $\rho = +0.87$ against the zoo GA and +0.89 against the zoo CMA-ES, over 41 champions), so the comparisons in the main text do not hinge on it. It does flatter them: 0 of the 41 beat the published GA champion (the best scores -0.04), and of the 10 above parity against the baseline, 5 also beat the CMA-ES policy (Table 10). The published GA champion is a single export its author chose to release, not an unselected endpoint. The CMA-ES policy, trained only against the baseline, is the stronger of the two by the baseline’s measure yet loses head to head to the GA champion (+0.30 for the GA): specialising against the yardstick, in one comparison.

Table 10: Final champions against the 2015 baseline (held out) and the two slimevolleygym zoo policies, points per episode, by condition.

condition	runs	vs 2015 baseline	vs zoo GA	vs zoo CMA-ES	beat zoo GA	beat zoo CMA-ES
control (Ha 2020 GA)	6	-0.15	-0.98	-0.10	0/6	2/6
archive as test, full span	6	-1.01	-1.72	-1.25	0/6	1/6
archive as parent, $p=0.25$	6	-4.10	-4.34	-3.95	0/6	1/6
archive as parent, $p=0.50$	1	-4.84	-4.99	-4.87	0/1	0/1
archive as parent, full span	3	-4.84	-4.98	-4.92	0/3	0/3
generational GA (Ha 2015)	6	-2.00	-3.83	-1.47	0/6	1/6
self-play ES	6	-2.08	-3.34	-2.41	0/6	1/6
$\sigma = 0.05$	3	-0.17	-0.62	+0.10	0/3	2/3
$\sigma = 0.20$	3	-1.66	-2.79	-2.08	0/3	0/3
population 32	1	-4.86	-4.99	-4.88	0/1	0/1

A.9 Unequal power (exploratory)

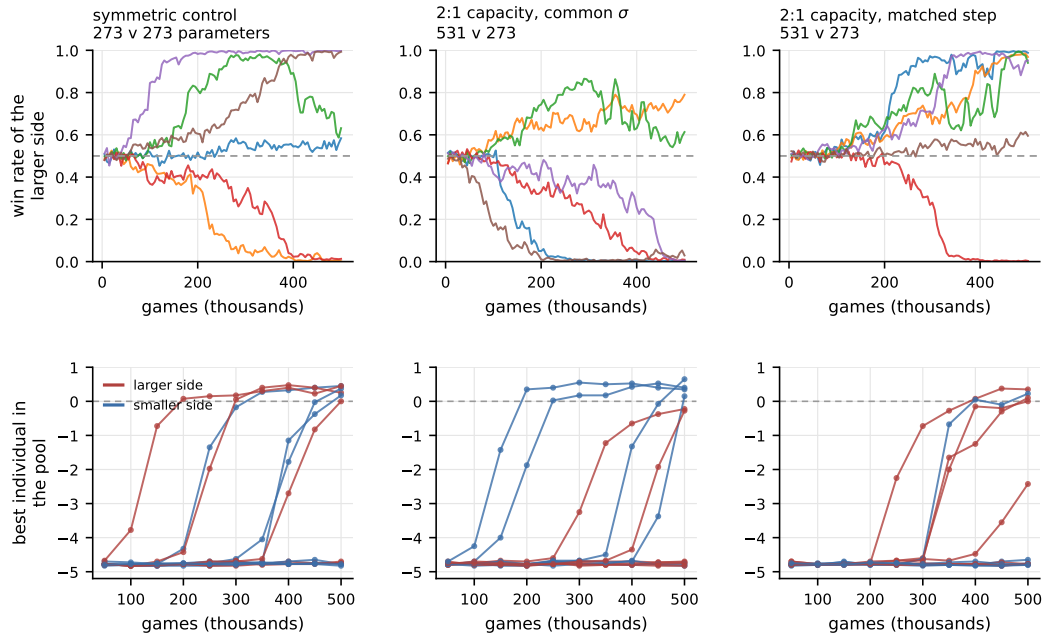


Figure 11: Two-population runs (exploratory, Section 4). Top: the larger side’s win rate in cross-population games, one line per seed. Bottom: the best member of each pool against the 2015 policy (red: larger side, blue: smaller).

Table 11: Two-population outcomes (exploratory). Win rate over the last 50,000 games; “best member” is the best individual in each pool at the end, against the 2015 policy.

condition	seeds	larger side wins cross-play	larger side’s pool, best member	smaller side’s pool, best member	runs where only one side’s pool learned
symmetric control (273 v 273)	6	0.62 (range 0.01–1.00); larger side ahead in 4/6	−2.35	−2.30	3/6
2:1 capacity, common σ	6	0.02 (range 0.00–0.75); larger side ahead in 2/6	−4.71	+0.25	4/6
2:1 capacity, matched step norm	6	0.95 (range 0.00–0.99); larger side ahead in 5/6	−1.21	−4.76	3/6

A.10 Every condition

Tables 12–15 give every metric for every condition, each condition’s comparison with the control, the within-run tournaments and the timing of the phase change.

Table 12: Every condition: endpoint and best checkpoint (held out), level, volatility and drawdown of the last 100,000 games, share of checkpoints above parity, median first-parity time.

condition	runs	final (held out)	peak (held out)	mean, last 100k	volatility	drawdown	above parity	median first parity
control (Ha 2020 GA)	6	−0.15 ± 0.27	+0.32 ± 0.06	−0.32 ± 0.23	0.67	0.60	26%	192k (6/6)
archive as test, full span	6	−1.01 ± 0.83	−0.55 ± 0.86	−1.06 ± 0.78	0.51	0.52	18%	195k (4/6)
archive as parent, p=0.25	6	−4.10 ± 0.74	−3.94 ± 0.90	−4.14 ± 0.70	0.12	0.24	2%	365k (1/6)
archive as parent, p=0.50	1	−4.84 ± —	−4.84 ± —	−4.85 ± —	0.01	0.06	0%	— (0/1)
archive as parent, full span	3	−4.84 ± 0.00	−4.83 ± 0.01	−4.84 ± 0.00	0.01	0.05	0%	— (0/3)
generational GA (Ha 2015)	6	−2.00 ± 0.82	−0.68 ± 0.83	−1.61 ± 0.70	0.63	0.70	3%	345k (5/6)
self-play ES	6	−2.08 ± 0.94	−1.56 ± 0.99	−2.46 ± 0.93	0.21	0.18	7%	398k (2/6)
sigma = 0.05	3	−0.17 ± 0.23	+0.34 ± 0.07	−0.29 ± 0.15	0.69	0.64	26%	150k (3/3)
sigma = 0.20	3	−1.66 ± 1.60	−1.44 ± 1.70	−1.76 ± 1.54	0.38	0.36	11%	290k (2/3)
population 32	1	−4.86 ± —	−4.84 ± —	−4.85 ± —	0.01	0.05	0%	— (0/1)

Table 13: Each condition against the control, exact two-sided Mann–Whitney.

condition	metric	difference	Cliff’s δ	exact p
archive as test, full span	final_holdout	−0.859	−0.28	0.485
archive as test, full span	late_mean	−0.745	−0.22	0.589
archive as test, full span	volatility	−0.157	−0.22	0.589
archive as test, full span	drawdown	−0.083	−0.11	0.818
archive as test, full span	above_parity	−0.087	−0.33	0.370
archive as parent, p=0.25	final_holdout	−3.951	−0.94	0.004
archive as parent, p=0.25	late_mean	−3.828	−0.94	0.004
archive as parent, p=0.25	volatility	−0.557	−0.83	0.015
archive as parent, p=0.25	drawdown	−0.366	−0.67	0.065
archive as parent, p=0.25	above_parity	−0.245	−0.94	0.004
archive as parent, full span	final_holdout	−4.688	−1.00	0.024
archive as parent, full span	late_mean	−4.528	−1.00	0.024
archive as parent, full span	volatility	−0.659	−1.00	0.024
archive as parent, full span	drawdown	−0.550	−1.00	0.024
archive as parent, full span	above_parity	−0.263	−1.00	0.024
generational GA (Ha 2015)	final_holdout	−1.852	−0.67	0.065
generational GA (Ha 2015)	late_mean	−1.299	−0.78	0.026
generational GA (Ha 2015)	volatility	−0.046	+0.00	1.000
generational GA (Ha 2015)	drawdown	+0.103	+0.00	1.000
generational GA (Ha 2015)	above_parity	−0.230	−0.83	0.015
self-play ES	final_holdout	−1.933	−0.33	0.394
self-play ES	late_mean	−2.145	−0.33	0.394
self-play ES	volatility	−0.464	−0.72	0.041
self-play ES	drawdown	−0.420	−0.83	0.015
self-play ES	above_parity	−0.197	−0.83	0.013
sigma = 0.05	final_holdout	−0.018	−0.22	0.714
sigma = 0.05	late_mean	+0.026	−0.44	0.381
sigma = 0.05	volatility	+0.017	+0.00	1.000
sigma = 0.05	drawdown	+0.034	+0.11	0.905
sigma = 0.05	above_parity	−0.003	−0.11	0.905
sigma = 0.20	final_holdout	−1.509	−0.33	0.548
sigma = 0.20	late_mean	−1.445	−0.44	0.381
sigma = 0.20	volatility	−0.294	−0.44	0.381
sigma = 0.20	drawdown	−0.238	−0.33	0.548
sigma = 0.20	above_parity	−0.150	−0.78	0.095

Table 14: Within-run checkpoint tournaments.

condition	runs	ρ (Elo, training time)	cyclic triads	undecided pairs
control (Ha 2020 GA)	6	+0.74	1/499 (0.2%)	10.2
archive as test, full span	6	+0.79	0/451 (0.0%)	12.8
archive as parent, p=0.25	6	+0.35	17/224 (7.6%)	20.8
archive as parent, p=0.50	1	+0.02	0/15 (0.0%)	26.0
archive as parent, full span	3	+0.06	9/29 (31.0%)	31.0
generational GA (Ha 2015)	6	+0.60	2/485 (0.4%)	12.3
self-play ES	6	+0.69	3/424 (0.7%)	14.3
sigma = 0.05	3	+0.91	0/239 (0.0%)	11.3
sigma = 0.20	3	+0.62	1/175 (0.6%)	15.7
population 32	1	+0.01	2/6 (33.3%)	29.0

Table 15: Phase-change timing per condition.

condition	runs	reached long rallies	internal transition (median, range)	first parity (median, range)	lag (median)
control (Ha 2020 GA)	6	6/6	120k (55k–415k)	192k (85k–440k) (6/6)	58k
archive as test, full span	6	5/6	150k (115k–190k)	195k (145k–260k) (4/6)	62k
archive as parent, p=0.25	6	1/6	80k (80k–80k)	365k (365k–365k) (1/6)	285k
archive as parent, p=0.50	1	0/1	never	never (0/1)	—
archive as parent, full span	3	0/3	never	never (0/3)	—
generational GA (Ha 2015)	6	5/6	205k (160k–420k)	345k (290k–480k) (5/6)	105k
self-play ES	6	4/6	372k (320k–490k)	398k (385k–410k) (2/6)	70k
sigma = 0.05	3	3/3	120k (80k–250k)	150k (145k–405k) (3/3)	65k
sigma = 0.20	3	2/3	228k (215k–240k)	290k (275k–305k) (2/3)	62k
population 32	1	0/1	never	never (0/1)	—
<i>reference run (1 run, real environment)</i>	1	1/1	<i>104k</i>	<i>172k</i>	<i>68k</i>

B The compiled environment

The environment, the evolved policy and the 2015 baseline were transcribed statement for statement into a numba-compiled module, including behaviour that looks like a bug but changes trajectories (for example, observations are not refreshed on a frame in which a point is scored). Three deviations are deliberate: the random-number generator family, the way two distinct population members are drawn, and the floating-point details of the network’s forward pass. The last can change a trajectory only by flipping the sign of a network output, because the game reads only whether each output is positive.

Per game. Driven from one stream of serve velocities, both implementations agree on every ball and agent position, every rally end, every score and every episode length in all 200 paired games (304,812 steps), across 4 populations chosen to exercise different parts of the state space (Table 16). The compiled version is 26 times faster on one core.

Per trajectory. Continuing the reference run’s committed population snapshot at 324,000 games for another 176,000 games, 2 compiled continuations put 54–60% of their checkpoints above parity, against 44% for the reference environment over the same games (Table 17). Independent continuations diverge because the algorithm is stochastic; they land in the same band.

Per distribution. The reference run’s transition times and best score lie inside the compiled control’s seed distribution (§3.2, Table 2).

Table 16: Bit-level validation of the compiled environment.

scenario	paired games	identical score	identical length	identical trajectory	max abs deviation	env steps compared
random vs random	50	50/50	50/50	50/50	0	31,971
champion vs random	50	50/50	50/50	50/50	0	60,465
champion vs champion	50	50/50	50/50	50/50	0	62,376
champion vs 2015 baseline	50	50/50	50/50	50/50	0	150,000

Table 17: Continuations of the reference run’s population snapshot.

continuation	games added	final score	checkpoints above parity
compiled, resume_s900	176,000	−2.82	19/35
compiled, resume_s901	176,000	+0.41	21/35
reference environment	176,000	+0.01	78/176

C Reproducing the results

Every number in this paper is regenerated from the run files by the repository’s scripts; with the pinned requirements installed, the sequence is

```
python test_repo.py                # self-test, incl. bit-level checks
python validate_fastvolley.py      # per-game validation
python run_experiments.py          # the single-population matrix
python run_asymmetric.py           # the two-population block
python train_ga_selfplay.py --resume # the reference run, slimevolleygym
python eval_reference.py
python analyze_matrix.py --holdout  # metrics and held-out re-scoring
python coevolution_analysis.py --within --across --proxy
python reexport.py                 # promotion rules
python resume_fast.py              # compiled continuations
python make_tables.py && python make_figures.py
paper/build.sh
```

Runs whose files exist are skipped, so the matrix can be resumed after an interruption. Each analysis script records the hash of every raw file it read and of its own output; `make_tables.py -check` fails if any table or number in the paper or the README no longer follows from the files on disk, and a continuous-integration job runs it, with the self-test, on every change. Rerunning the analysis reproduced the committed files: the reference curve and the compiled continuations byte for byte, the held-out scores, the within-run tournaments, the export-rule check and the promotion-rule analysis in every reported value. Every change to the design after the first run started is recorded, with what was known at the time, in the decision log shipped with the data; it includes the archive-as-test bug, its rerun and the comparison of the two.